Vulnerability Notes

Business Logic Vulnerabilities

Concepts, Attack Techniques, Detection, and Defenses

Reference
PortSwigger Web Security Academy

Prepared by
Youssef Reda

Contents

Business Logic Vulnerabilities **1**
- Definition and Core Concepts . . . 1
- Business Logic vs. Technical Vulnerabilities . . . 1
- Unexpected Input and Application States . . . 1
- Detection and Testing . . . 1
- Example: Transaction Amount Manipulation . . . 1

How Business Logic Vulnerabilities Arise **2**
- Flawed Assumptions . . . 2
- Client-Side Validation Only . . . 2
- Unexpected Workflow and Application State . . . 2
- Interactions Between Components . . . 2
- Responsibility Gaps and Poor Documentation . . . 2
- How a Logic Flaw Develops . . . 2

Impact of Business Logic Vulnerabilities **3**

Common Business Logic Flaws **4**
- Excessive Trust in Client-Side Controls . . . 4
  - Testing . . . 4
- Failing to Handle Unconventional Input . . . 4
  - Boundary and Type Testing . . . 4
- Making Flawed Assumptions About User Behavior . . . 5
  - Trusted Users Won't Always Remain Trustworthy . . . 5
  - Users Won't Always Supply Mandatory Input . . . 5
  - Users Won't Always Follow the Intended Sequence . . . 5
- Domain-Specific Flaws . . . 6
  - Discounts and State Inconsistency . . . 6
  - User-Controlled Conditions and Algorithms . . . 6
  - Combining Legitimate Functions . . . 6
- Providing an Encryption Oracle . . . 6
  - Encryption, Encoding, and Decryption . . . 6
  - When It Becomes Dangerous . . . 6
  - Lab: Authentication Bypass via Encryption Oracle . . . 7
  - Testing and Prevention . . . 7
- Email Address Parser Discrepancies . . . 8
  - Encoded-Word Syntax . . . 8
  - UTF-7 and the Lab . . . 8
  - Lab: Bypassing Access Controls Using Email Address Parsing Discrepancies . . . 8

How to Prevent Business Logic Vulnerabilities **9**

Understand the Application Domain 9
Avoid Implicit Assumptions 9
Validate Inputs and State Server-Side 9
Use Clear Development Practices 9

Business Logic Vulnerabilities

Definition and Core Concepts

Business logic vulnerabilities are flaws in an application's design or implementation that let attackers misuse legitimate functionality to produce unintended results. They occur when business rules, constraints, or workflows are missing, inconsistent, or incorrectly enforced. Business rules define what the application should allow. Examples include requiring payment before confirming an order, restricting coupon use, enforcing inventory limits, and allowing refunds only for eligible purchases. A logic flaw exists when the application fails to enforce such a rule.

Business Logic vs. Technical Vulnerabilities

Technical vulnerabilities, such as SQL injection, XSS, and command injection, often follow recognizable technical patterns. Business logic vulnerabilities depend on application-specific rules and workflows. An application can be protected against common technical attacks while still allowing payment bypasses, discount abuse, or unauthorized actions.

Unexpected Input and Application States

Attackers may submit missing, negative, oversized, or unexpected values; modify parameters between requests; repeat requests; skip workflow steps; or combine legitimate features in unintended ways. Pay particular attention to transaction-critical values such as prices, quantities, discounts, balances, shipping costs, user IDs, roles, and permissions.

Detection and Testing

These flaws are difficult to detect automatically because testing requires an understanding of intended application behavior. First map the normal workflow and business rules, then test whether the server enforces them when requests or application state are manipulated.

1. Identify the intended workflow and required conditions.
2. List business rules, limits, and assumptions.
3. Skip, repeat, or reorder steps where safe to test.
4. Modify important values and omit optional or required parameters.
5. Combine legitimate features and check their interactions.
6. Verify actual state changes, not only the HTTP response.
7. Assess whether the result violates a rule or provides an unauthorized advantage.

Example: Transaction Amount Manipulation

If a product costs $100, the server must verify that the amount paid matches the valid order total before confirming the order. If an attacker changes a client-controlled payment amount to $1 and the server still confirms the $100 order, the application has failed to enforce a transaction rule.

How Business Logic Vulnerabilities Arise

Flawed Assumptions

Developers may assume users will use only the interface, submit valid input, complete every step, follow the expected sequence, and never replay or directly invoke requests. These assumptions become vulnerabilities when the server relies on them instead of enforcing the rules itself.

Client-Side Validation Only

Client-side controls improve usability but are not a security boundary. An attacker can intercept and modify requests using tools such as Burp Suite. Any security-sensitive constraint must be revalidated server-side.

```
POST /purchase
quantity=100
```

If the interface limits quantity to 10 but the server accepts the modified request, the restriction is ineffective.

Unexpected Workflow and Application State

A server must verify that prerequisites are satisfied at the time an operation is performed. For example, it should not trust that a user reviewed the cart or paid merely because the browser normally sends requests in that order. Recheck current prices, cart contents, payment status, and authorization before finalizing a transaction.

Interactions Between Components

Components may behave correctly in isolation while their combination creates a flaw. For example, a coupon may be limited to one use and refunds may be legitimate, but an inconsistent refund workflow might restore neither the coupon state nor the usage count correctly, allowing reuse.

Responsibility Gaps and Poor Documentation

Different developers may each assume another component performs a required validation, leaving the check unimplemented. Unclear code and undocumented assumptions make these gaps harder to detect. Document workflows, data flows, business rules, component responsibilities, and side effects.

How a Logic Flaw Develops

```
Flawed assumption
      |
      v
Missing or inconsistent validation
      |
      v
Unexpected user action or state
      |
      v
Application violates a business rule
```

Impact of Business Logic Vulnerabilities

The severity depends on the affected functionality and the realistic consequences. Possible impacts include:

- **Authentication bypass:** Accessing accounts or protected functionality without satisfying required checks.
- **Privilege escalation:** Performing actions reserved for higher-privileged users.
- **Financial loss and fraud:** Manipulating prices, transfers, balances, discounts, refunds, or payments.
- **Data exposure:** Accessing sensitive information through improperly enforced workflows or permissions.
- **Business disruption:** Causing unauthorized transactions, operational problems, unexpected costs, or reputational damage.

Assess which functionality is affected, what an attacker can achieve, whether other attack surfaces become accessible, and the likely impact on users and the business.

Common Business Logic Flaws

Excessive Trust in Client-Side Controls

The application relies on browser-side restrictions to enforce rules. Attackers can modify client-controlled parameters or send requests directly.

Testing

1. Identify values restricted by the interface, such as price, quantity, role, or discount.
2. Intercept the request and modify the relevant value.
3. Submit the request and verify that the server independently enforces the rule.
4. Check the resulting application state for unauthorized changes.

Client-side validation is useful for usability; server-side validation is mandatory for security.

Failing to Handle Unconventional Input

Applications may validate only data type or format without checking whether a value makes sense under the business rules. For example, a numeric transfer amount may still be invalid if it is negative or exceeds a permitted limit.

Boundary and Type Testing

Test values that are relevant to the field and its rules:

- Negative, zero, minimum, and maximum values.
- Values just below or above known boundaries.
- Very large or small values.
- Empty values, omitted parameters, unexpected types, and unusual formats.

Data-type validation does not guarantee business-rule validation. For example, a check that only verifies whether a transfer amount is less than the balance may accept a negative amount:

```
if ($transferAmount <= $currentBalance) {
    // Complete transfer
}
```

The server must also enforce that the amount is positive and satisfies all transaction rules. Use Burp Repeater to compare normal requests with carefully modified requests, then verify state changes.

Making Flawed Assumptions About User Behavior

Do not assume users will supply every parameter, remain trustworthy after an initial check, or follow the intended workflow. Validate each sensitive operation against the current server-side state.

Trusted Users Won't Always Remain Trustworthy

A user or value validated earlier may later become untrusted or change. Recheck authorization and relevant data when performing each sensitive action; do not treat an earlier check as permanent permission for all later operations.

Users Won't Always Supply Mandatory Input

Applications may behave unexpectedly when a parameter is missing or empty. Test the original value, an empty value, and a completely omitted parameter separately.

1. Record the normal request and response.
2. Replace the parameter with an empty value.
3. Remove the parameter entirely.
4. Compare responses, redirects, and any state changes.
5. Repeat for relevant URL parameters, POST fields, and cookies.

The server should reject missing required input explicitly instead of entering an unintended code path or relying on client-side required-field checks.

Users Won't Always Follow the Intended Sequence

Test whether users can skip, repeat, reorder, or directly access workflow steps. For example, an application must verify that a user has completed the required authentication stages before granting access to a protected endpoint.

1. Complete the normal workflow and identify requests for each step.
2. Replay a later request without completing its prerequisites.
3. Repeat requests to check for duplicate operations.
4. Return to earlier steps and modify data after later steps have occurred.
5. Verify that each endpoint checks the current state and required permissions.

Use Burp Proxy and Repeater to inspect and replay requests. Forced browsing or direct endpoint access may reveal missing server-side workflow checks. Errors can also disclose assumptions about required steps or parameters.

Domain-Specific Flaws

These flaws arise from incorrect assumptions about the application's business domain or algorithms. They require understanding how the system is supposed to work and how state changes affect related features.

Discounts and State Inconsistency

Discounts, refunds, cancellations, inventory, and order state must remain consistent across the entire workflow. Test whether a legitimate operation, such as a refund, incorrectly enables a discount or coupon to be reused.

User-Controlled Conditions and Algorithms

Identify values that influence business decisions and determine whether the user can manipulate them. Understand the calculation or decision process, then test edge cases and interactions between features. A value that appears harmless in one component may have unexpected effects in another.

Combining Legitimate Functions

Individually valid features can produce an invalid result when combined. Test related features together and verify that business rules remain enforced across purchases, cancellations, refunds, subscriptions, and account changes.

Providing an Encryption Oracle

An encryption oracle accepts plaintext and returns ciphertext. It is not inherently vulnerable; the risk arises when an attacker can submit chosen plaintext and reuse the resulting ciphertext in another feature that trusts its decrypted contents.

Encryption, Encoding, and Decryption

- **Encoding/decoding:** Changes or restores a representation, such as Base64 or URL encoding; it does not provide confidentiality.
- **Encryption/decryption:** Uses a cryptographic algorithm and key to transform plaintext into ciphertext and recover it.
- A cookie may contain URL-encoded, Base64-encoded ciphertext. Reverse these representation layers before examining the binary ciphertext; decoding alone does not decrypt it.

```
URL decode -> Base64 decode -> ciphertext
                                  |
                                  v
                     Decryption requires key
                                  |
                                  v
                              Plaintext
```

When It Becomes Dangerous

The oracle may be exploitable if another component accepts compatible ciphertext and uses the decrypted data for authentication, authorization, transactions, or other sensitive decisions. Compatibility depends on more than the algorithm or key: mode, IV/nonce, authentication tag, padding, and expected format may matter. A decryption oracle may reveal plaintext or only observable results, such as whether decryption or validation succeeded. It can help identify a data format, but it is not always necessary if the expected plaintext is already known.

Lab: Authentication Bypass via Encryption Oracle

In the lab, a notification feature encrypts user-controlled input, while the persistent-login mechanism processes a `stay-logged-in` cookie. The notification error adds a fixed prefix, and examining the relevant behavior reveals that the persistent-login plaintext contains a username and timestamp:

```
wiener:timestamp
```

The encryption uses 16-byte blocks. The error prefix is 23 bytes, so adding nine characters makes the unwanted prefix occupy 32 bytes:

$$23 + 9 = 32 = 2 \times 16 \text{ bytes}$$

After obtaining the ciphertext, the lab solution removes the first 32 bytes and tests the remaining ciphertext in the persistent-login context. In a hex editor, 32 bytes correspond to 64 hexadecimal characters, not 32 hex characters. This behavior depends on the lab's encryption mode and is not a universal property of encryption. If the modified cookie is accepted as administrator login data, it can bypass authentication. The underlying issue is that attacker-influenced ciphertext is accepted in a security-sensitive context without adequate integrity protection and validation.

Testing and Prevention

1. Identify encrypted cookies, tokens, and parameters.
2. Determine the expected plaintext structure where possible.
3. Check whether other components accept ciphertext generated by the oracle.
4. Assess whether decrypted values influence identity, permissions, or transactions.
5. Verify integrity protection and independent server-side authorization checks.

Prefer authenticated encryption (such as an appropriate AEAD scheme), verify authentication tags, avoid unnecessary encryption/decryption oracles, and never treat encryption alone as proof that data is trustworthy.

Email Address Parser Discrepancies

A parser discrepancy occurs when different application components interpret the same email address differently. For example, registration validation may inspect one representation while the email-processing component decodes or parses it differently, allowing the confirmation message to reach an attacker-controlled address.

Encoded-Word Syntax

Email headers may use the Encoded-Word form:

```
=?charset?encoding?encoded-text?=
```

The charset identifies the character encoding, the encoding field commonly uses `Q` or `B`, and the final field contains encoded text. For example:

```
=?iso-8859-1?q?=61=62=63?=
```

represents `abc`. Different parsers or decoding stages may handle encoded input differently.

UTF-7 and the Lab

In UTF-7, sequences such as `&AEA-` can represent `@`, and `&ACA-` can represent a space. The lab demonstrates that some Encoded-Word variations are blocked while a UTF-7 variation is processed differently. The attack relies on a mismatch between the address checked by registration validation and the recipient derived by email processing. A crafted address can appear to satisfy the trusted-domain restriction to one component while being interpreted as an attacker-controlled recipient by another.

Lab: Bypassing Access Controls Using Email Address Parsing Discrepancies

1. Confirm that a registration address outside the permitted domain is blocked.
2. Test encoded variants and observe whether validation treats them differently.
3. Use the lab's UTF-7-based address construction with the supplied exploit-server identifier.
4. Check the exploit server's email client for the confirmation message.
5. Follow the confirmation link, access the administrative functionality, and delete the designated user to complete the lab.

The lesson is to validate the canonical email address that will actually be used, and to apply consistent parsing and normalization across validation, account storage, and email delivery.

How to Prevent Business Logic Vulnerabilities

Understand the Application Domain

Developers and testers must understand the business rules, workflows, and constraints the application is intended to enforce.

Avoid Implicit Assumptions

Do not assume users will follow the intended workflow or that other components always provide valid data. Identify assumptions about server-side state and verify them before proceeding.

Validate Inputs and State Server-Side

Check that input is sensible and permitted by the business rules. Before sensitive operations, verify current state, authorization, required workflow steps, and transaction values. Client-side validation alone is insufficient.

Use Clear Development Practices

- Maintain design documents and data flows for transactions and workflows.
- Write clear, readable code and document unavoidable complexity.
- Define which component is responsible for each validation.
- Review dependencies and side effects if inputs or component behavior are manipulated.
- Analyze the root cause of each discovered flaw to prevent recurrence.